

```
# -*- coding: utf-8 -*-
"""
True joint-level impedance control for the real Piper, using the MIT interface.
```

Why this file

~~~~~  
``impedance\_control.py`` implements a virtual-spring on top of ``EndPoseCtrl``. That call goes through Piper's on-board high-stiffness position servo, so the end-effector cannot actually be back-driven by external force: even though the script caps ``|x\_cmd - x\_actual|`` to ``F\_max/Kp``, the on-board servo treats that gap as a position error to crush at full torque.

To get true compliance, this file uses ``JointMitCtrl`` (Motor Impedance Torque), Piper's low-level joint interface:

```
JointMitCtrl(motor_num, pos_ref, vel_ref, kp, kd, t_ref)
```

with small ``kp`` / ``kd``. Each joint becomes a spring-damper with stiffness ``kp`` (N m / rad), so:

- \* In free space: joint error drives the joint toward ``pos\_ref`` -> the arm walks to the target.
- \* Push it: external force easily back-drives the joint (the spring is weak).
- \* Hold against obstacle: torque saturates at ``kp \* (q\_ref - q\_actual)`` -> bounded.

Cartesian target is converted to a joint target via Pinocchio IK (using the URDF at ``piper\_ros/src/piper\_description/urdf/piper\_description.urdf``).

Run env: conda env ``piper\_work\_env`` (which has ``pin`` installed).  
~~~~~

```
import argparse
import math
import os
import select
import sys
import termios
import time
import tty
from contextlib import contextmanager
from dataclasses import dataclass, field
from typing import List, Optional, Tuple

import numpy as np

try:
    import pinocchio as pin
except ImportError as e:
    raise ImportError(
        "pinocchio is required. In the piper_work_env conda env run:\n"
        "    pip install pin\n"
        f"Original error: {e}"
    )
```

```
from piper_sdk import C_PiperInterface_V2
```

```
# ----- constants
```

```
_DEFAULT_URDF = os.path.join(
    os.path.expanduser("~"), "piper_robot",
    "piper_ros/src/piper_description/urdf/piper_description.urdf",
)
```

```
# MotionCtrl_2 mask for MIT mode (move_mode=0x04, mit_mode=0xAD).
_MIT_CTRL_2 = (0x01, 0x04, 0, 0xAD)
```

```
# rad -> 0.001 deg, matches the demo convention.
_RAD_TO_MDEG = 1000.0 * 180.0 / math.pi
```

```
# m -> 0.001 mm (the unit EndPoseCtrl expects).
_M_TO_UMM = 1.0e6
```

```
# ===== non-blocking stdin ==
```

```
@contextmanager
def _raw_stdin():
    """Put stdin into cbreak mode so single keypresses are readable without
    waiting for Enter. Restores terminal state on exit. No-op when stdin is
    not a TTY."""
    if not sys.stdin.isatty():
        yield False
        return
    fd = sys.stdin.fileno()
    old = termios.tcgetattr(fd)
    try:
        tty.setcbreak(fd)
        yield True
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old)
```

```
def _key_pressed() -> Optional[str]:
    """Return the most recent buffered key (single char) or None."""
    if not sys.stdin.isatty():
        return None
    last = None
    while True:
        r, _, _ = select.select([sys.stdin], [], [], 0)
        if not r:
            break
        last = sys.stdin.read(1)
    return last
```

```
# ===== kinematics ==
```

```
class PiperKinematics:
    """Pinocchio wrapper for Piper FK / IK.

    Locks the gripper joints (`joint7`, `joint8`) so the reduced model has
    exactly the 6 controllable joints. The EE frame is added at `joint6` with
    identity offset, matching `piper_ros/.../piper_pinocchio.py`.
    """
```

```
def __init__(self, urdf_path: str = _DEFAULT_URDF):
    if not os.path.isfile(urdf_path):
        raise FileNotFoundError(f"URDF not found: {urdf_path}")

    full_model = pin.buildModelFromUrdf(urdf_path)
    locked = [
        full_model.getJointId(name)
        for name in ("joint7", "joint8")
        if full_model.existJointName(name)
    ]
    self.model = pin.buildReducedModel(
        full_model, locked, np.zeros(full_model.nq)
    )
    self.data = self.model.createData()

    if not self.model.existFrame("ee"):
        ee_frame = pin.Frame(
            "ee",
            self.model.getJointId("joint6"),
            pin.SE3.Identity(),
            pin.FrameType.OP_FRAME,
        )
        self.model.addFrame(ee_frame)
        self.data = self.model.createData()
    self.ee_id = self.model.getFrameId("ee")
```

```
assert self.model.nq == 6, f"reduced model has nq={self.model.nq}, expected 6"
print(f"[mit-kin] loaded URDF, nq={self.model.nq}, ee frame id={self.ee_id}")
```

```
def fk(self, q: np.ndarray) -> pin.SE3:
    pin.forwardKinematics(self.model, self.data, q)
    pin.updateFramePlacements(self.model, self.data)
    return self.data.oMf[self.ee_id].copy()
```

```
def ik(
    self,
    target_se3: pin.SE3,
    q_init: np.ndarray,
    max_iters: int = 300,
    eps: float = 1e-4,
    damping: float = 1e-4,
    step: float = 0.6,
) -> Tuple[np.ndarray, bool]:
    """Damped least-squares IK in local (body) frame. Returns (q, success)."""
    q = q_init.copy()
    I6 = np.eye(6)
    for _ in range(max_iters):
        pin.forwardKinematics(self.model, self.data, q)
        pin.updateFramePlacements(self.model, self.data)
        cur = self.data.oMf[self.ee_id]
        err = pin.log(cur.actInv(target_se3)).vector
        if np.linalg.norm(err) < eps:
            return q, True
        J = pin.computeFrameJacobian(
            self.model, self.data, q, self.ee_id, pin.ReferenceFrame.LOCAL
        )
        dq = J.T @ np.linalg.solve(J @ J.T + damping * I6, err)
        q = pin.integrate(self.model, q, step * dq)
    return q, False
```

```
# ===== params ==
@dataclass
class MitImpedanceParams:
    """Per-joint MIT controller gains and behaviour knobs."""

    mit_kp: List[float] = field(default_factory=lambda: [12.0, 18.0, 15.0, 10.0, 8.0, 6.0])
    mit_kd: List[float] = field(default_factory=lambda: [1.2, 1.5, 1.2, 0.8, 0.6, 0.5])

    # Single soft<->stiff knob. kp_eff = mit_kp * stiffness, kd_eff = mit_kd * sqrt(stiffness).
    stiffness: float = 1.0

    # Cap per-joint position error to bound max torque. 0 disables.
    max_joint_err_rad: float = 0.35 # ~20 deg

    rate_hz: float = 200.0
    ramp_s: float = 2.0

    # Re-assert MIT mode every cycle (cheap; matches the demo).
    reassert_mode_each_cycle: bool = True
```

```
# ===== letter path ==
class LetterPath:
    """2D letter path in the XZ plane, defined in letter-local coordinates.
```

Coordinate convention

```
-----
* (0, 0) is the letter's start point (top-left by convention).
* +x → right
* +z → up (so descending strokes use negative z values)
* The Y dimension is fixed in 3D; it is supplied by the world origin at
  evaluation time and never changes within a letter.
```

Usage

Build the path by chaining `.line_to()` and `.arc()` calls, then pass the instance to `PiperMitImpedance.run_letter_mit()`.

Example – letter U (`arc_radius = height = 5 cm`):

```
path = (LetterPath(v_speed=0.03)
        .line_to(0.00, -0.05)          # left arm down
        .arc(0.05, -0.05, 0.05, 180, 360) # bottom semicircle
        .line_to(0.10, 0.00))          # right arm up
.....

def __init__(self, v_speed: float = 0.02):
    self.v_speed = float(v_speed)
    self._segs: List[Tuple] = [] # each: (duration, callable(t)->np.ndarray[x,z])
    self._total_time: float = 0.0
    self._pos = np.zeros(2, dtype=float) # current pen position (x, z)

# ----- builders
def line_to(self, x: float, z: float) -> "LetterPath":
    """Straight stroke to letter-frame point (x, z)."""
    p0 = self._pos.copy()
    p1 = np.array([x, z], dtype=float)
    dist = float(np.linalg.norm(p1 - p0))
    if dist < 1e-9:
        return self
    dur = dist / self.v_speed
    self._segs.append((dur, lambda t, a=p0, b=p1, d=dur: a + (t / d) * (b - a)))
    self._total_time += dur
    self._pos = p1.copy()
    return self

def arc(
    self,
    cx: float, cz: float,
    radius: float,
    start_deg: float, end_deg: float,
) -> "LetterPath":
    """Circular arc centred at (cx, cz) with given radius.

    Angles use math convention: 0° = +x, 90° = +z, 180° = -x, 270° = -z.
    The sweep direction follows the sign of (end_deg - start_deg).
    """
    span_rad = math.radians(end_deg - start_deg)
    arc_len = abs(span_rad) * radius
    if arc_len < 1e-9:
        return self
    dur = arc_len / self.v_speed
    a0 = math.radians(start_deg)
    a1 = math.radians(end_deg)
    c = np.array([cx, cz], dtype=float)
    self._segs.append((
        dur,
        lambda t, c=c, r=radius, a0=a0, a1=a1, d=dur: (
            c + r * np.array([math.cos(a0 + (t / d) * (a1 - a0)),
                             math.sin(a0 + (t / d) * (a1 - a0))])
        ),
    ))
    self._total_time += dur
    self._pos = c + radius * np.array([math.cos(a1), math.sin(a1)])
    return self

def ellipse_arc(
    self,
    cx: float, cz: float,
    rx: float, rz: float,
    start_deg: float, end_deg: float,
) -> "LetterPath":
    """Elliptical arc centred at (cx, cz) with semi-axes rx (+x) and rz (+z).

    Angles use math convention: 0° = +x, 90° = +z.
```

Arc length is approximated numerically for duration calculation.

```
"""
span_rad = math.radians(end_deg - start_deg)
n = 200
thetas = np.linspace(math.radians(start_deg), math.radians(end_deg), n + 1)
dxs = -rx * np.sin(thetas[:-1]) * (span_rad / n)
dzs = rz * np.cos(thetas[:-1]) * (span_rad / n)
arc_len = float(np.sum(np.sqrt(dxs ** 2 + dzs ** 2)))
if arc_len < 1e-9:
    return self
dur = arc_len / self.v_speed
a0 = math.radians(start_deg)
a1 = math.radians(end_deg)
self._segs.append((
    dur,
    lambda t, cx=cx, cz=cz, rx=rx, rz=rz, a0=a0, a1=a1, d=dur: np.array([
        cx + rx * math.cos(a0 + (t / d) * (a1 - a0)),
        cz + rz * math.sin(a0 + (t / d) * (a1 - a0)),
    ]),
))
self._total_time += dur
self._pos = np.array([cx + rx * math.cos(a1), cz + rz * math.sin(a1)])
return self
```

----- query

```
@property
def total_time(self) -> float:
    return self._total_time
```

```
@property
def end_local(self) -> np.ndarray:
    """End point in letter-local (x, z)."""
    return self._pos.copy()
```

----- eval

```
def __call__(self, t: float, origin: np.ndarray) -> np.ndarray:
    """Absolute 3D world position at time t into the letter stroke.

    origin : 3D start point of the letter (e.g. top-left corner in world).
    Returns np.ndarray shape (3,).
    """
    xz = self._eval_xz(t)
    return np.array([origin[0] + xz[0], origin[1], origin[2] + xz[1]])

def _eval_xz(self, t: float) -> np.ndarray:
    for dur, fn in self._segs:
        if t <= dur:
            return fn(t)
        t -= dur
    _, fn = self._segs[-1] # past the end -> hold last point
    return fn(self._segs[-1][0])
```

===== controller ==

```
class PiperMitImpedance:
    """Standalone MIT-mode impedance controller for the Piper arm."""

    def __init__(
        self,
        can_name: str = "can0",
        urdf_path: str = _DEFAULT_URDF,
        home_joints_rad: Optional[List[float]] = None,
        start_z_offset_m: float = 0.0,
    ):
        self.can_name = can_name
        self.piper = C_PiperInterface_V2(can_name)
        self.home_joints_rad: List[float] = (
            list(home_joints_rad) if home_joints_rad is not None else [0.0] * 6
        )
        self.start_z_offset_m: float = float(start_z_offset_m)
```

```

self.kin = PiperKinematics(urdf_path)

# ----- setup
def connect_and_enable(self) -> None:
    self.piper.ConnectPort(start_thread=True, piper_init=True)
    print(f"[mit] connected on {self.can_name}")

    while not self.piper.EnablePiper():
        time.sleep(0.01)
    time.sleep(0.1)
    print("[mit] enabled")

    self.piper.GripperCtrl(0, 1000, 0x02, 0)
    self.piper.GripperCtrl(0, 1000, 0x01, 0)

    self._set_ctrl_mode_can()

def _set_ctrl_mode_can(self) -> bool:
    """Transition to CAN control mode (mirrors piper_control.py:set_ctrl_mode2can)."""
    ctrl_mode_str = {0x00: "STANDBY", 0x01: "CAN_CTRL", 0x02: "TEACH"}
    cur = self.piper.GetArmStatus().arm_status.ctrl_mode
    print(f"[mit] ctrl_mode={ctrl_mode_str.get(cur, hex(cur))}")

    if cur == 0x00:
        print("[mit] STANDBY -> CAN")
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)
        time.sleep(1.0)
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    elif cur == 0x02:
        print("[mit] TEACH -> STANDBY -> CAN")
        self.piper.MotionCtrl_1(0x02, 0, 0)
        time.sleep(1.0)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)
        time.sleep(1.0)
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    elif cur == 0x01:
        print("[mit] already in CAN")
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    else:
        print(f"[mit] unexpected ctrl_mode={hex(cur)}; trying CAN")
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)
        time.sleep(1.0)
        while not self.piper.EnablePiper():
            time.sleep(0.01)
        self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00)

    self.piper.GripperCtrl(0, 1000, 0x02, 0)
    self.piper.GripperCtrl(0, 1000, 0x01, 0)

    t0 = time.time()
    while time.time() - t0 < 5.0:
        if self.piper.GetArmStatus().arm_status.ctrl_mode == 0x01:
            print("[mit] CAN control mode confirmed")
            return True
        time.sleep(0.05)

    print("[mit] WARNING: failed to confirm CAN control mode within 5 s")
    return False

def ensure_can_ready(self) -> bool:
    return self._set_ctrl_mode_can()

```

```

# ----- I0 helpers
def read_ee_pose_raw(self) -> Tuple[int, int, int, int, int, int]:
    ep = self.piper.GetArmEndPoseMsgs().end_pose
    return (ep.X_axis, ep.Y_axis, ep.Z_axis, ep.RX_axis, ep.RY_axis, ep.RZ_axis)

def read_ee_position_m(self) -> np.ndarray:
    x, y, z, *_ = self.read_ee_pose_raw()
    return np.array([x, y, z], dtype=float) / _M_TO_UMM

def _wait_reached(self, timeout_s: float, poll_s: float = 0.05) -> bool:
    """Block until motion_status == 0x00 ('reached') or timeout.

    Sleeps 0.15 s before polling so the firmware can register the new
    command; without this delay the function may return True immediately
    because the previous move just finished (motion_status is still 0x00).
    """
    time.sleep(0.15)
    t0 = time.time()
    while time.time() - t0 < timeout_s:
        if self.piper.GetArmStatus().arm_status.motion_status == 0x00:
            return True
        time.sleep(poll_s)
    return False

def read_joint_rad(self) -> np.ndarray:
    js = self.piper.GetArmJointMsgs().joint_state
    j_mdeg = [js.joint_1, js.joint_2, js.joint_3,
              js.joint_4, js.joint_5, js.joint_6]
    return np.array([math.radians(v / 1000.0) for v in j_mdeg], dtype=float)

# ----- motion
def goto_home(self, speed: int = 30, timeout_s: float = 10.0) -> bool:
    """Move to self.home_joints_rad via MOVE_J."""
    if not self.ensure_can_ready():
        print("[mit] first ensure_can_ready failed -> retrying after 1 s")
        time.sleep(1.0)
        self.ensure_can_ready()

    self._print_arm_state("entering goto_home")

    j = [int(round(v * _RAD_TO_MDEG)) for v in self.home_joints_rad]
    print(f"[mit] goto_home (joints rad): {[round(v, 4) for v in self.home_joints_rad]}")
    self.piper.MotionCtrl_2(0x01, 0x01, speed, 0x00) # CAN + MOVE_J
    self.piper.JointCtrl(j[0], j[1], j[2], j[3], j[4], j[5])

    reached = self._wait_reached(timeout_s)
    if reached:
        print(f"[mit] home reached; EE (m) = {self.read_ee_position_m()}")
    else:
        print(f"[mit] WARNING: home not confirmed within {timeout_s:.1f}s")
    return reached

def goto_start(self, speed: int = 30, timeout_s: float = 10.0) -> bool:
    """Move to home, then offset Z by start_z_offset_m via MOVE_P."""
    if not self.goto_home(speed=speed, timeout_s=timeout_s):
        print("[mit] aborting goto_start: home not reached")
        return False

    dz = self.start_z_offset_m
    if dz == 0.0:
        return True

    X, Y, Z, RX, RY, RZ = self.read_ee_pose_raw()
    Z_new = Z + int(round(dz * _M_TO_UMM))
    print(f"[mit] goto_start: Z offset {dz*1000:.0f} mm -> Z={Z_new}")
    self.piper.MotionCtrl_2(0x01, 0x00, 100, 0x00) # CAN + MOVE_P
    self.piper.EndPoseCtrl(X, Y, Z_new, RX, RY, RZ)
    time.sleep(0.01)

```

```

reached = self._wait_reached(timeout_s)
if reached:
    print(f"[mit] start reached; EE (m) = {self.read_ee_position_m()}")
else:
    print(f"[mit] WARNING: start not confirmed within {timeout_s:.1f}s")
return reached

# ----- MIT
def _send_mit(self, q_ref: np.ndarray, kp: List[float], kd: List[float]) -> None:
    for i in range(6):
        self.piper.JointMitCtrl(i + 1, float(q_ref[i]), 0.0, float(kp[i]), float(kd[i]), 0.0)

def _print_arm_state(self, tag: str) -> None:
    st = self.piper.GetArmStatus().arm_status
    ctrl = {0x00: "STANDBY", 0x01: "CAN", 0x02: "TEACH"}.get(st.ctrl_mode, hex(st.ctrl_mode))
    mfeed = {0x00: "MOVE_P", 0x01: "MOVE_J", 0x02: "MOVE_L",
             0x03: "MOVE_C", 0x04: "MIT"}.get(st.mode_feed, hex(st.mode_feed))
    ms = {0x00: "REACHED", 0x01: "MOVING"}.get(st.motion_status, hex(st.motion_status))
    print(f"[mit-debug] {tag}: ctrl_mode={ctrl} mode_feed={mfeed} "
          f"motion_status={ms} err_code={st.err_code}")

def _exit_mit_to_can(self, base_kp: List[float], base_kd: List[float]) -> None:
    """Smooth teardown from MIT mode back to CAN/MOVE_J.

    The core trick: read the actual joint angles while MIT gains are still
    zero (arm is floating), then hand those exact angles to the MOVE_J
    position controller. The controller sees ~zero error on its first cycle
    and applies no sudden torque, preventing the disable-on-switch.
    """
    self._print_arm_state("before exit-MIT teardown")

    # 1. Ramp gains to zero so joints float before mode switch.
    try:
        for alpha in (0.6, 0.3, 0.1, 0.0):
            q_now = self.read_joint_rad()
            kp_step = [k * alpha for k in base_kp]
            kd_step = [k * math.sqrt(max(alpha, 0.0)) for k in base_kd]
            self.piper.MotionCtrl_2(*MIT_CTRL_2)
            self._send_mit(q_now, kp_step, kd_step)
            time.sleep(0.05)
        q_now = self.read_joint_rad()
        self.piper.MotionCtrl_2(*MIT_CTRL_2)
        self._send_mit(q_now, [0.0] * 6, [0.0] * 6)
        time.sleep(0.1)
    except Exception as e:
        print(f"[mit-debug] gain-rampdown error (ignored): {e!r}")

    # 2. Snapshot actual joint angles while the arm is still floating.
    q_actual = self.read_joint_rad()
    j_actual = [int(round(v * _RAD_TO_MDEG)) for v in q_actual]
    print(f"[mit-debug] handoff joints (rad): {[round(v, 4) for v in q_actual]}")

    # 3. Switch to CAN + MOVE_J at low speed, target = current position.
    # Sending the target before and after EnablePiper keeps error ≈ 0.
    for _ in range(3):
        self.piper.MotionCtrl_2(0x01, 0x01, 10, 0x00) # MOVE_J, 10 % speed
        self.piper.JointCtrl(*j_actual)
        time.sleep(0.08)

    t0 = time.time()
    while not self.piper.EnablePiper():
        if time.time() - t0 > 2.0:
            print("[mit-debug] EnablePiper timed out after MIT exit")
            break
        time.sleep(0.01)

    # 4. Reaffirm target after enable so the controller latches onto it.
    self.piper.MotionCtrl_2(0x01, 0x01, 10, 0x00)
    self.piper.JointCtrl(*j_actual)
    time.sleep(0.3)

```

```

self._print_arm_state("after exit-MIT teardown")

def run_to_target_mit(
    self,
    target_m: np.ndarray,
    params: MitImpedanceParams = MitImpedanceParams(),
    duration_s: Optional[float] = 30.0,
    quit_joints_rad: Optional[List[float]] = None,
) -> None:
    """Drive EE toward target_m and hold it with joint-level compliance.

    quit_joints_rad: if provided, the arm moves to these joint angles after
    exiting MIT mode (on 'q', duration reached, or Ctrl-C).
    """
    target_m = np.asarray(target_m, dtype=float).reshape(3)

    q_start = self.read_joint_rad()
    ee_start = self.kin.fk(q_start)
    target_pose = pin.SE3(ee_start.rotation, target_m)
    q_target, ok = self.kin.ik(target_pose, q_start)
    if not ok:
        print(f"[mit] WARNING: IK did not converge to target {target_m}. "
              f"Using best-effort q_target = {q_target}.")
    print(f"[mit] q_start (rad): {q_start.round(4)}")
    print(f"[mit] q_target (rad): {q_target.round(4)}")
    print(f"[mit] ee_start (m) : {ee_start.translation.round(4)}")
    print(f"[mit] target (m) : {target_m.round(4)}")

    if params.stiffness <= 0.0:
        raise ValueError(f"stiffness must be > 0, got {params.stiffness}")
    scale_p = float(params.stiffness)
    scale_d = math.sqrt(scale_p)
    kp_eff = [k * scale_p for k in params.mit_kp]
    kd_eff = [k * scale_d for k in params.mit_kd]
    tau_max_eff = [k * params.max_joint_err_rad for k in kp_eff]
    print(f"[mit] stiffness={scale_p:.2f} "
          f"kp_eff={[round(k, 2) for k in kp_eff]} "
          f"kd_eff={[round(k, 2) for k in kd_eff]} "
          f"tau_max={[round(t, 2) for t in tau_max_eff]} Nm")

    print("[mit] entering MIT mode")
    self.piper.MotionCtrl_2(*MIT_CTRL_2)
    time.sleep(0.05)
    self._send_mit(q_start, kp_eff, kd_eff)
    time.sleep(0.05)

    dt = 1.0 / params.rate_hz
    t0 = time.time()
    last_log = -1.0

    try:
        with _raw_stdin() as kb_active:
            if kb_active:
                print("[mit] press 'q' at any time to abort")
            while True:
                loop_start = time.time()
                now = loop_start - t0

                key = _key_pressed()
                if key in ("q", "Q"):
                    print("[mit] 'q' pressed -> returning to zero")
                    break

                q = self.read_joint_rad()

                if params.ramp_s > 0 and now < params.ramp_s:
                    alpha = now / params.ramp_s
                    q_d = q_start + alpha * (q_target - q_start)
                else:
                    q_d = q_target.copy()

```

```

        if params.max_joint_err_rad > 0:
            err = q_d - q
            np.clip(err, -params.max_joint_err_rad, params.max_joint_err_rad, out=err)
            q_d_clamped = q + err
        else:
            q_d_clamped = q_d

        if params.reassert_mode_each_cycle:
            self.piper.MotionCtrl_2(*MIT_CTRL_2)
        self._send_mit(q_d_clamped, kp_eff, kd_eff)

        if now - last_log >= 0.5:
            last_log = now
            x_act = self.kin.fk(q).translation
            err_cart = float(np.linalg.norm(target_m - x_act))
            err_joint = q_target - q
            tau_approx = np.array(kp_eff) * (q_d_clamped - q)
            print(f"[mit] t={now:5.2f}s ee_err={{err_cart*1000:6.1f}}mm "
                  f"|q_err|max={{float(np.max(np.abs(err_joint))):5.3f}}rad "
                  f"|tau_p|max={{float(np.max(np.abs(tau_approx))):5.2f}}Nm "
                  f"(press q to quit)")

        if duration_s is not None and now >= duration_s:
            print(f"[mit] duration reached ({duration_s:.1f}s)")
            break

        sleep_left = dt - (time.time() - loop_start)
        if sleep_left > 0:
            time.sleep(sleep_left)

    finally:
        print("[mit] exiting MIT mode -> CAN/MOVE_P")
        try:
            self._exit_mit_to_can(params.mit_kp, params.mit_kd)
        except Exception as e:
            print(f"[mit] teardown failed: {e!r}")

        if quit_joints_rad is not None:
            print(f"[mit] going to quit position: {[round(v, 4) for v in quit_joints_rad]}")
            _saved = self.home_joints_rad
            self.home_joints_rad = list(quit_joints_rad)
            self.goto_home(speed=30, timeout_s=10.0)
            self.home_joints_rad = _saved

def run_trajectory_mit(
    self,
    traj_fn,
    ramp_target_m: np.ndarray,
    params: MitImpedanceParams = MitImpedanceParams(),
    duration_s: Optional[float] = 30.0,
    quit_joints_rad: Optional[List[float]] = None,
) -> None:
    """Drive EE along a time-varying Cartesian trajectory with joint-level compliance.

    During ramp_s the arm interpolates (joint-space) toward ramp_target_m.
    After ramp_s, traj_fn(t_after_ramp) is called each cycle to get the
    Cartesian target; IK is solved with warm-starting from the previous result.

    traj_fn: callable(float) -> np.ndarray shape (3,)
        Returns desired EE position in metres at time t seconds after the
        ramp phase ends.
    ramp_target_m: Cartesian position to ramp toward before the trajectory starts.
    quit_joints_rad: joint angles to move to after exiting MIT mode.
    """
    ramp_target_m = np.asarray(ramp_target_m, dtype=float).reshape(3)

    q_start = self.read_joint_rad()
    ee_start = self.kin.fk(q_start)

```

```

ramp_pose = pin.SE3(ee_start.rotation, ramp_target_m)
q_ramp, ok = self.kin.ik(ramp_pose, q_start)
if not ok:
    print(f"[mit] WARNING: IK did not converge to ramp target {ramp_target_m}. "
          f"Using best-effort.")
print(f"[mit] q_start      (rad): {q_start.round(4)}")
print(f"[mit] q_ramp_target(rad): {q_ramp.round(4)}")
print(f"[mit] ee_start      (m) : {ee_start.translation.round(4)}")
print(f"[mit] ramp_target   (m) : {ramp_target_m.round(4)}")

if params.stiffness <= 0.0:
    raise ValueError(f"stiffness must be > 0, got {params.stiffness}")
scale_p = float(params.stiffness)
scale_d = math.sqrt(scale_p)
kp_eff = [k * scale_p for k in params.mit_kp]
kd_eff = [k * scale_d for k in params.mit_kd]
tau_max_eff = [k * params.max_joint_err_rad for k in kp_eff]
print(f"[mit] stiffness={scale_p:.2f} "
      f"kp_eff={round(k, 2) for k in kp_eff} "
      f"kd_eff={round(k, 2) for k in kd_eff} "
      f"tau_max={round(t, 2) for t in tau_max_eff} Nm")

print("[mit] entering MIT mode (trajectory)")
self.piper.MotionCtrl_2(*_MIT_CTRL_2)
time.sleep(0.05)
self._send_mit(q_start, kp_eff, kd_eff)
time.sleep(0.05)

dt = 1.0 / params.rate_hz
t0 = time.time()
last_log = -1.0
# Warm-start IK from the ramp target after ramp phase.
q_ik_prev = q_ramp.copy()

try:
    with _raw_stdin() as kb_active:
        if kb_active:
            print("[mit] press 'q' at any time to abort")
        while True:
            loop_start = time.time()
            now = loop_start - t0

            key = _key_pressed()
            if key in ("q", "Q"):
                print("[mit] 'q' pressed -> returning to zero")
                break

            q = self.read_joint_rad()

            if params.ramp_s > 0 and now < params.ramp_s:
                alpha = now / params.ramp_s
                q_d = q_start + alpha * (q_ramp - q_start)
                cart_target = ramp_target_m
            else:
                t_traj = now - params.ramp_s
                cart_target = np.asarray(traj_fn(t_traj), dtype=float).reshape(3)
                tgt_pose = pin.SE3(ee_start.rotation, cart_target)
                q_ik, ik_ok = self.kin.ik(tgt_pose, q_ik_prev)
                if ik_ok:
                    q_ik_prev = q_ik
                    q_d = q_ik_prev.copy()

            if params.max_joint_err_rad > 0:
                err = q_d - q
                np.clip(err, -params.max_joint_err_rad, params.max_joint_err_rad, out=err)
                q_d_clamped = q + err
            else:
                q_d_clamped = q_d

            if params.reassert_mode_each_cycle:

```

```

        self.piper.MotionCtrl_2(*MIT_CTRL_2)
        self._send_mit(q_d_clamped, kp_eff, kd_eff)

        if now - last_log >= 0.5:
            last_log = now
            x_act = self.kin.fk(q).translation
            err_cart = float(np.linalg.norm(cart_target - x_act))
            tau_approx = np.array(kp_eff) * (q_d_clamped - q)
            t_traj_display = max(0.0, now - params.ramp_s)
            print(f"[mit] t={now:5.2f}s  t_traj={t_traj_display:5.2f}s  "
                  f"ee_err={err_cart*1000:6.1f}mm  "
                  f"target(m)={cart_target.round(4)}  "
                  f"|tau_p|max={float(np.max(np.abs(tau_approx))):5.2f}Nm  "
                  f"(press q to quit)")

        if duration_s is not None and now >= duration_s:
            print(f"[mit] duration reached ({duration_s:.1f}s)")
            break

        sleep_left = dt - (time.time() - loop_start)
        if sleep_left > 0:
            time.sleep(sleep_left)

    finally:
        print("[mit] exiting MIT mode -> CAN/MOVE_P")
        try:
            self._exit_mit_to_can(params.mit_kp, params.mit_kd)
        except Exception as e:
            print(f"[mit] teardown failed: {e!r}")

        if quit_joints_rad is not None:
            print(f"[mit] going to quit position: {[round(v, 4) for v in quit_joints_rad]}")
            _saved = self.home_joints_rad
            self.home_joints_rad = list(quit_joints_rad)
            self.goto_home(speed=30, timeout_s=10.0)
            self.home_joints_rad = _saved

def run_letter_mit(
    self,
    letter: LetterPath,
    forward_m: np.ndarray,
    params: MitImpedanceParams = MitImpedanceParams(),
    repeat: int = 1,
    duration_s: float = 300.0,
    quit_joints_rad: Optional[List[float]] = None,
) -> None:
    """Run the full sequence: ramp forward -> draw letter -> return to start.

    Parameters
    -----
    letter      : LetterPath instance defining the stroke in XZ-local coords.
    forward_m   : 3D offset from current EE position to the letter origin
                  (top-left corner), e.g. np.array([0, 0.10, 0]) for 10 cm in Y.
    params      : MIT impedance gains / timing.
    repeat      : how many times to draw the letter (ramp only once; letter ->
                  return cycles repeat times, then holds at x0).
    duration_s  : how long to hold at x0 after all repetitions finish
                  (press 'q' to exit early).
    quit_joints_rad : joint angles to move to after MIT mode ends.
    """
    x0 = self.read_ee_position_m()
    letter_origin = x0 + np.asarray(forward_m, dtype=float)

    end_xz = letter.end_local
    letter_end_3d = np.array([
        letter_origin[0] + end_xz[0],
        letter_origin[1],
        letter_origin[2] + end_xz[1],
    ])

```

```

return_dist = float(np.linalg.norm(letter_end_3d - x0))
t_return = return_dist / letter.v_speed if return_dist > 1e-6 else 0.1
t_letter = letter.total_time
t_cycle = t_letter + t_return      # one full letter + return
t_all = t_cycle * max(repeat, 1) # total trajectory time

print(f"[mit] letter_origin={letter_origin.round(4)} "
      f"letter_end={letter_end_3d.round(4)} "
      f"t_letter={t_letter:.1f}s t_return={t_return:.1f}s "
      f"repeat={repeat} t_all={t_all:.1f}s")

def full_traj(t: float) -> np.ndarray:
    if t >= t_all:      # all reps done → hold at x0
        return x0.copy()
    t_in_cycle = t % t_cycle
    if t_in_cycle < t_letter:
        return letter(t_in_cycle, letter_origin)
    t_r = t_in_cycle - t_letter
    frac = t_r / t_return
    return letter_end_3d + frac * (x0 - letter_end_3d)

self.run_trajectory_mit(
    full_traj,
    ramp_target_m=letter_origin,
    params=params,
    duration_s=duration_s,
    quit_joints_rad=quit_joints_rad,
)

```

===== demo

```

if __name__ == "__main__":
    """

```

Motion sequence:

- 1) connect + enable;
- 2) go to INIT position: joint1 = home_j1, joints 2-6 = 0;
- 3) go to HOME (= impedance start): full home_joints_rad;
- 4) MIT impedance hold toward target for duration_s;
- 5) return to INIT position on exit.

Usage:

```

python impedance_control_mit.py --letter U    # default
python impedance_control_mit.py --letter C
python impedance_control_mit.py --letter L
python impedance_control_mit.py --letter A    # downward-facing C (arch)
"""

```

```

parser = argparse.ArgumentParser(
    description="Draw a letter with MIT impedance control."
)
parser.add_argument(
    "--letter",
    choices=["U", "C", "L", "A"],
    default="U",
    help=(
        "Letter to draw in the XZ plane: "
        "U, C, L, or A (A = downward-facing C / arch shape). "
        "Default: U."
    ),
)
args = parser.parse_args()

```

----- positions

```

# Keep this in sync with piper_control.py home_joints_rad.
_HOME_JOINTS_RAD = [1.57, 0.866, -0.960, 0.0, 0.182, 0]

```

Initial and final resting pose: only joint 1 matches home, rest at zero.

```

_INIT_JOINTS_RAD = [_HOME_JOINTS_RAD[0], 0.0, 0.0, 0.0, 0.0, 0.0]

```

----- setup

```

time.sleep(5.0)

```

```

ctrl = PiperMitImpedance(
    can_name="can0",
    urdf_path=_DEFAULT_URDF,
    home_joints_rad=_HOME_JOINTS_RAD,
    start_z_offset_m=0,
)
ctrl.connect_and_enable()

# Step 1: go to initial position
print("[mit] moving to init position...")
ctrl.home_joints_rad = _INIT_JOINTS_RAD
ctrl.goto_home(speed=30, timeout_s=10.0)
input("[mit] reached init position. Press Enter to continue to home...")

# Step 2: go to home = impedance start
print("[mit] moving to home (start) position...")
ctrl.home_joints_rad = _HOME_JOINTS_RAD
ctrl.goto_start(speed=30, timeout_s=10.0)
input("[mit] reached home position. Press Enter to start drawing...")

# ----- letter
# All paths are defined in local XZ coords (origin = pen start).
# +x → right +z → up (down strokes use negative z)
# Letter height / width: ~10 cm.
print(f"[mit] drawing letter: {args.letter}")
if args.letter == "U":
    # Three straight segments: left arm down → bottom → right arm up. Width: 6 cm, height: 8 cm.
    letter = (LetterPath(v_speed=0.02)
        .line_to(0.00, -0.08) # left arm down
        .line_to(0.06, -0.08) # bottom across
        .line_to(0.06, 0.00)) # right arm up
elif args.letter == "C":
    # Origin (0,0) = top-right corner. Three straight segments. Width: 8 cm, height: 8 cm.
    letter = (LetterPath(v_speed=0.02)
        .line_to(-0.04, 0.00) # top stroke left
        .line_to(-0.04, -0.08) # left side down
        .line_to(0.00, -0.08)) # bottom stroke right
elif args.letter == "L":
    # Vertical stroke down → horizontal stroke right. Width: 6 cm, height: 8 cm.
    letter = (LetterPath(v_speed=0.02)
        .line_to(0.00, -0.08) # down 8 cm
        .line_to(0.06, -0.08)) # right 6 cm
elif args.letter == "A":
    # Descend 8 cm, then draw a narrow-tall downward-facing C (n arch).
    # rx=0.03 → width 6 cm; rz=0.07 → arch rises 7 cm above base.
    letter = (LetterPath(v_speed=0.02)
        .line_to(0.00, -0.08) # down 8 cm
        .ellipse_arc(0.03, -0.08, 0.03, 0.07, 180, 0)) # narrow-tall arch

# ----- params
params = MitImpedanceParams(
    mit_kp=[10.0, 8.0, 8.0, 8.0, 6.0, 6.0],
    mit_kd=[0.8, 0.6, 0.6, 0.6, 0.5, 0.5],
    stiffness=0.3,
    max_joint_err_rad=0.35,
    rate_hz=200.0,
    ramp_s=2.0,
)

# ----- run
try:
    ctrl.run_letter_mit(
        letter,
        forward_m=np.array([0.0, 0.10, 0.0]), # advance 10 cm in +Y
        params=params,
        repeat=1, # draw the letter twice
        duration_s=300.0, # hold at x0 after; press 'q' to exit
        quit_joints_rad=_INIT_JOINTS_RAD,
    )

```

```
except KeyboardInterrupt:  
    print("[mit] interrupted")
```